INFINITE · AUDIOVISUAL NODE WORKSTATION

The Field

Language Manual

Everything you need to write real Field kernels and build real devices — the domains, the syntax, the declarations, and the six node types that host them.

Covers **Field v1** | Style **direct, example-driven** | Audience **device builders & AI assistants**

One primitive, five domains

A Field kernel is a short body of code, run once per element of a domain. Everything else — whether it becomes a shader, a geometry deformer, or a DSP effect — falls out of which identifiers it touches and which node hosts it.

- `graph` — edit-time, mounts other nodes
- `element` — once per point/vertex
- `sample` — once per audio sample
- `frame` — once per rendered frame
- `pixel` — once per pixel, on the GPU

BEFORE YOU START

What Field Is

Infinite is a node-based audiovisual workstation — you build patches by wiring boxes together on a canvas. Most boxes do one fixed thing. Field is different: it's a small text language that lets **you** write what a box does, instead of picking from a menu.

The one idea underneath all of Field is the **kernel**: a short body of code run once per element of some domain, automatically, forever. Write `col = vec3(uv.x, uv.y, 0.5)` and it runs once per pixel, 60 times a second, on the GPU — a shader. Write `P.y += sin(P.x * 4.0 + t) * 0.2` and it runs once per point in a mesh — a geometry deformer. Write `out = in * 0.5` and it runs once per audio sample — a DSP effect. You never say where it runs; Field infers that from what identifiers your code touches.

Every example in this manual is valid, runnable Field code written against the six node types that actually ship in Infinite today.

CONTENTS

Twelve Chapters

CHAPTER TWO

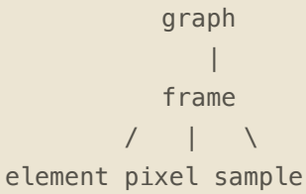
The Five Domains

A domain is "what counts as one element." Four of the five map to real Field node types — the fifth, `frame`, has no node of its own.

DOMAIN	RUNS…	HOSTED BY	PRODUCES
graph	once, at edit time (create / Regenerate)	Field Graph	mounts & wires other real Infinite nodes
frame	once per rendered frame (~60/s)	hoisted inside any of the other five, for shared values	one value shared across all elements that frame
element	once per point/vertex per frame (60×N/s)	Field Modifier, Field Primitive	mesh/point-cloud geometry & per-point color
pixel	once per pixel per frame, on the GPU	Field Pixel	a 2D texture (color)
sample	once per audio sample per voice (48,000/s)	Field Effect, Field Synth	an audio signal

The domain lattice

This is the shape that governs what can see what. `element`, `pixel`, and `sample` are mutually incomparable peers — an element kernel cannot read a pixel or sample value directly, in either direction. The only way to move a value between them is up through `frame` and back down — Chapter 6.



Which node hosts which domain

- `element` kernels live in the **Field Modifier** node (deforms geometry already on the canvas) or the **Field Primitive** node (generates points from nothing)
- `pixel` kernels live in the **Field Pixel** node
- `sample` kernels live in the **Field Effect** node (an `in` pin to process) or the **Field Synth** node (note-driven, `freq` / `gate`, no `in` pin)
- `graph` kernels live in the **Field Graph** node

See Chapter 9 for what each of these six nodes actually does.

CHAPTER THREE

Core Syntax

Field reads like a small C-family language. Here's everything that isn't obvious from that alone.

Bare names, no sigil

Every identifier — reserved names like `P`, `uv`, `in`, `out`, and your own `param` / `state` / `attrib` names — is written bare, exactly like a normal variable. Unlike Houdini VEX's `@P` style, Field has no sigil at all.

```
P.y += sin(P.x * 4.0 + t) * 0.2
```

Types & swizzles

The type keywords are `float`, `int`, `bool`, `vec2`, `vec3`, `vec4`. There is no `string` type inside a kernel body. Swizzles work like GLSL: `.x .y .z .w` and `.r .g .b .a`, 1–4 letters — but the two letter sets don't mix in one swizzle (`.xg` isn't valid; use `.xy` or `.rg`).

Operators

Arithmetic `+ - * / %`, comparison `< <= > >= == !=`, boolean `&& || !`, and `^` for exponentiation (`a ^ b` = `pow(a,b)`) — not XOR. There are no bitwise operators.

Comments & broadcast

`//` starts a line comment. A scalar auto-broadcasts against a vector: `P += 0.1` adds to every component, no `vec3(0.1,0.1,0.1)` needed.

Branching with if()

`if(cond, a, b)` is a function — a ternary that always returns a value, not an if/else statement. In `pixel` kernels it lowers to a GLSL `mix()`, so both branches are evaluated on the GPU every time; in `element` / `sample` kernels (CPU, one path per element/voice) it's ordinary, cheap control flow.

```
hit = if(P.y > threshold, 1.0, 0.0)
```

CHAPTER FOUR

Declarations — attrib, param, state

Three keywords cover everything a kernel needs to remember or expose: a per-element scratch value, a UI knob, and a value that reads back one cycle late.

attrib — a value that lives per-element

element

```
attrib float heat = 0
heat = (P.y + 1.0) * 0.5
Cd = vec3(heat, 0.2, 1.0 - heat)
```

Element attribute ramp driving vertex color

Declares a per-element scratch value, initialized once; whatever you assign into it that frame is what the rest of the kernel body sees.

param — a value that becomes a UI knob

`param <type> <name> = <default> [<lo>, <hi>]` declares a real, modulatable knob — it shows up on the node's faceplate, can be dragged, automated, and cabled to a modulator.

```
param float speed = 2.0 [0.1, 10.0]
```

> **Punctuation:** in `pixel` kernels every statement ends with a semicolon (`param float speed = 2.0 [0.1, 10.0];`). In `element` and `sample`, statements are newline-terminated with no semicolon.

state — this frame reads last frame's value

```
state float y = 0
y = y * 0.999 + in * 0.1
out = y
```

One-pole recursive smoother in the sample domain

`state` declares a cell read as its value from the **previous** evaluation; whatever you assign becomes what the next evaluation sees — which is why `y = y * 0.999 + …` is legal and not an infinite loop. It's the only way to loop a value back inside a kernel. State survives hot-reload (editing the code doesn't reset an unrelated filter) and saves with the patch.

CHAPTER FIVE

Reserved Words per Domain

Each domain seeds identifiers that already mean something. Redeclaring one with `attrib` / `param` / `state` is a compile error — not a warning.

DOMAIN	RESERVED NAMES	NOTES
element	`P N Cd i count t dt`	`i` is the element index — pick another name for your own loop variable
pixel	`uv col t dt frame`	never redeclare `uv` / `col`
sample	`in out sr freq gate`	`freq` / `gate` are the current voice's note frequency and note-on state
graph	*(none)*	a one-shot script; uses `emit` / `connect` / `set` / `place` instead — Chapter 8

PI

`pi` is only available in a pure frame-domain expression. Inside `element` and `pixel` kernels, write out the numeric constant longhand: `3.14159265`.

```
p = uv * 3.14159265;
```

Nodal pattern coordinate scaling

CHAPTER SIX

Domain Transfer Operators

Since element, pixel, and sample can't see each other directly, Field gives you explicit operators to move data across — always by way of frame.

OPERATOR	DIRECTION	HOW YOU WRITE IT
reduce	many (one domain) → one (frame)	`reduce.rms(…)`, `reduce.max(…)`, etc.
broadcast	one (frame) → many (any domain)	nothing — assign a frame-hoisted value and it broadcasts automatically
map	one-per-element, within a domain	a language construct for per-element transforms
resample / downsample	rate-changing crossings	for the rarer case where you need a different rate, not just a different domain

Domain transfer examples

```
// element domain — a per-element hit, reduced to one frame-rate value:
hit = if(P.y > threshold, 1.0, 0.0)
output frame float chime = reduce.max(hit)
```

Boundary sensor: per-element crossing reduced to frame

```
// sample domain — the audio buffer's RMS in a band, reduced to one value:
output frame float bass = reduce.rms(in, 20.0, 160.0)
```

Audio band energy reduced to frame

`reduce` ships with `sum`, `mean`, `rms`, `rmsBandLimited`, `min`, and `max`. Going the other direction needs no keyword: reading a `param` that's fed by a frame-rate value elsewhere broadcasts to every pixel/element/sample for free, because a `param` read is already a per-domain broadcast.

reduce is a frame-rate operator — it works from inside an element or sample kernel to collapse many values into one. It isn't available inside a pixel kernel; an aggregate of pixel data has to arrive as a value computed elsewhere and passed in.

CHAPTER SEVEN

Randomness

`rand`, `noise`, and `sh` are pure functions of their arguments — same inputs, same output, always, so a kernel stays deterministic and re-runnable. There's no hidden global RNG state to seed or reset.

rand(min, max, speed, seed)

element

Takes 0 to 4 arguments — fewer arguments fall back to sane defaults.

```
n = rand(0, 1, 2.0, 0)
P += N * (n - 0.5) * 0.2
```

Normal displacement using smooth noise: 0 to 1, speed 2.0, seed 0

noise & sh

`noise` and `sh` follow the same pure-function pattern as `rand`, taking 0–4 arguments. Reach for `rand` first for most device work; use `noise` / `sh` when you want a smoother or shaped variant and want to compare results by ear or eye.

CHAPTER EIGHT

Building a Device

There are two ways to bundle Field kernels into something that behaves like one reusable device, rather than a single raw kernel box.

1 · Single-node dynamic pins

Any element, pixel, or sample kernel can declare extra pins right in its code (up to 16 combined input+output pins per kernel):

```
output frame float glow = abs(sin(t * speed))
```

This creates a real extra pin on the canvas node that can be wired with a cable, saves and loads with the patch, and survives undo. It's the cheapest way to add a second domain's worth of I/O to a device that's really one native node's behavior.

> **What a declared output pin carries today:** a fixed value of 0. For a value that genuinely reaches another node right now, read it off one of the node's built-in toggle outputs instead — **rms** on Field Effect/Field Synth, **publish** on Field Modifier/Field Primitive, or **state** on Field Pixel. Use a hand-declared output pin when you want the pin present and wired for later, and one of the three toggles when you need the value live today.

2 · Field Graph bundle ("Instrument Mode")

For a device needing more than one kernel body, a Field Graph node runs its code once, at edit time, using four functions to mount and wire real Infinite nodes for you:

```
emit("Type Name", k0, k1, ...) -> handle
connect(srcHandle, srcSlot, dstHandle, dstSlot)
set(handle, "paramName", value)
place(handle, x, y)
```

Live params on the graph node's own knobs forward into the emitted nodes, and the bundle's texture/audio pins forward correctly out to the rest of your patch. Toggle **Instrument Mode** to collapse the bundle into one box, or **Unpack to Canvas** to show the emitted nodes separately and rewire them by hand.

CHAPTER NINE

Field Node Reference

Field is exposed to the user as six node types, each hosting one domain's kernel. All six compile the same source-text grammar covered in Chapters 1–8 — the only thing that changes is which reserved names are in scope and which backend the kernel is compiled to.

NODE	DOMAIN	WHAT IT DOES
Field Modifier	element	Runs a per-vertex kernel over geometry already on the canvas, modifying `P`, `N`, `uv`, `Cd` and custom attributes.
Field Primitive	element	Generates procedural point geometry from scratch — no input required — using a per-element kernel (Circle, Spiral, Grid Lattice, Fibonacci Sphere, Helix, Torus Knot presets as starting points).
Field Effect	sample	Runs a kernel once per audio sample, on the audio thread. `in` is the audio input pin, `state` declares per-voice memory, `param` exposes a knob, `reduce.rms(in, loHz, hiHz)` publishes a band-limited RMS reading once per block.
Field Synth	sample	A polyphonic, note-driven synthesizer using the same sample-domain compiler as Field Effect, but with `freq` / `gate` in place of an `in` pin — a self-contained voice, no upstream audio required.
Field Pixel	pixel	Runs a kernel once per pixel, compiled to GLSL. `uv`, `col`, `res` are the reserved names; a `state` cell declared `[wrap]` plus offset reads like `A(uv + d)` support feedback and stencils.
Field Graph	graph	Runs a kernel once, at edit time, to build part of the patch itself — `emit` / `connect` / `set` / `place` spawn and wire real Infinite nodes. See Chapter 8.

Pick the node by domain first, then by role: element splits into **modify** (Field Modifier) vs. **generate** (Field Primitive); sample splits into **process** (Field Effect, has an `in` pin) vs. **generate** (Field Synth, note-driven). Pixel and graph each have one node.

CHAPTER TEN

Device Faceplate

Every Field node has two views: a compact body on the canvas, and a separate editor window for the code.

The two views

- **The compact node body** — shows the auto-generated `param` knobs, notice text, and an "Edit Field…" button. No source code is visible here.
- **The floating editor window**, opened by "Edit Field…", is where you read and write the kernel's text, inspect compilation status, and see a live preview.

What the faceplate gives you

- Knob rows laid out on Infinite's standard grid, so a Field node's knobs look and behave like every other node's — even with a variable-length, user-authored param list.
- An inline preview in the compact body: a live texture thumbnail on Field Pixel, a live scope on Field Effect and Field Synth, and a mini-viewport on Field Modifier and Field Primitive.
- Device save, import, and export controls inside the editor window, allowing you to manage device files without closing the code view.

CHAPTER ELEVEN

Saving & Sharing Devices — .infdev

Field devices save and load through a portable file format — plain JSON.

The .infdev shape

```
{
  "format": "infdev",
  "version": 1,
  "meta": {
    "name": "My Device",
    "author": "you",
    "description": "optional",
    "domain": "element"
  },
  "device": {
    "code": "param float amp = 0.8 [0.0, 3.0]\n...",
    "params": { "amp": 0.8 },
    "nodeSettings": { "maxElements": 65536, "generateCount": 64 }
  }
}
```

`nodeSettings` differs per domain — a Field Primitive device carries `maxElements` and, when it has no upstream input, `generateCount`.

Saving, loading, sharing

- Use the **Devices** dropdown plus Save/Export/Import on the node body to save a device or load one someone sent you.
- Drag an `.infdev` file onto the matching node type to load it directly.
- You can hand-author or hand-edit an `.infdev` file in a text editor — it's JSON with a `code` string field — as long as that field stays valid Field for the `domain` declared in `meta`.

CHAPTER TWELVE

Common Mistakes

The wrong/right pairs a fresh session — human or AI — hits first.

WRONG	WHY	RIGHT
`@P.y += 0.1`	Field has no sigil	`P.y += 0.1`
`param int voices = 8`	param only accepts float	`param float voices = 8.0`
using `i` as your own loop var in element	`i` is the reserved element index	use a different name, e.g. k
`col = vec3(uv.x, uv.y, 0.5)`	pixel statements need a semicolon	`col = vec3(uv.x, uv.y, 0.5);`
`x = pi * 2.0`	`pi` only works in a frame expression	`x = 3.14159265 * 2.0`
`n = rand(t, seed)`	the real signature is `rand(min, max, speed, seed)`	`n = rand(0, 1, 2.0, seed)`
wiring a declared output pin expecting a live value	it carries a fixed 0 today	use the node's rms/publish/state toggle instead
`.xg` swizzle	can't mix xyzw and rgba letter sets	`.xy` or `.rg`, not mixed
redeclaring `P, uv, in, out`	shadowing a reserved name is a compile error	pick a different name
expecting an element value inside a pixel kernel	element/pixel/sample can't see each other directly	reduce → frame → broadcast, explicitly

Quick Reference

Type keywords

`float int bool vec2 vec3 vec4`

Operators

`+ - * / % ^` arithmetic (^ = power)

`< <= > >= == !=` comparison

`&& || !` boolean

`+= -= *= /=` compound assign

Swizzles: `.x.y.z.w` / `.r.g.b.a`, 1–4 letters, not mixed

Built-in functions

1-arg: sin cos tan abs floor ceil round sign exp sqrt log fract length normalize

2-arg: min max mod fmod pow step distance dot cross atan2

3-arg: clamp lerp mix smoothstep if

0–4-arg: rand noise sh

DOMAIN	RESERVED	NODE(S)
element	`P N Cd i count t dt`	Field Modifier, Field Primitive
pixel	`uv col t dt frame`	Field Pixel
sample	`in out sr freq gate`	Field Effect, Field Synth
graph	(none — uses emit/connect/set/place)	Field Graph

The two device-building mechanisms

1. Single-node dynamic pins — `output` / `input` declarations inside a kernel. The pin is real and wireable; its value today is a fixed 0 — use the rms/publish/state toggle for a live value now.

2. Field Graph bundle ("Instrument Mode") — `emit` / `connect` / `set` / `place`, runs once at edit time, with live param forwarding into the emitted nodes.